Keeping objects valid

Today

Welcome to your next lesson: Keeping objects valid.

I'm preparing your personalized lesson based on what you've already learned, what comes next, and the best way to help you practice.

Step by step, you’re moving forward.

Imagine you have a Pet object with an age of -5 years 🐶

This makes no sense in the real world, but without extra care your class would happily store -5 and pretend it’s fine. Today is about stopping that: we’ll make our objects start life in a reasonable, safe state.

First idea for today:

Constructor validation = checking the values inside __init__ before you save them on self, and correcting them if needed.

Example in words (no code yet):

Input: age = -5
Check: “is this age valid?”

If valid → store it
If not valid → store a safe fallback (like 0)
Quick check for you:

If you had a Pet with age -3, which feels more responsible?

Store -3 as-is
Or change it to some safe value like 0 or 1

Just answer in words; then we'll turn this into Python in the next step.

Change it to a safe value

Exactly 🎯 — that's the core idea: don't let bad data slip in; fix it to something safe.

Now let's turn that into a tiny Python pattern.

Inside __init__, we can do this:

Look at the input (like age)
Check if it's valid (e.g. age >= 0)
If valid → use it
If not → use a safe fallback (like 0)

In simple code form, the shape looks like this (don't type yet, just read):

Python
```
if some_condition_about_input:
    self.attribute = input_value      # accepted
else:
    self.attribute = fallback_value   # safe fallback
```
Make sense so far: input → check → either keep it or replace with a safe fallback?

If yes, I'll next show you a concrete Pet example and then have you implement your own version in the editor.

Yes, that makes sense

Nice 👍

Let’s make this concrete with a Pet example.

Imagine this class:

Python

```
class Pet:
    def __init__(self, name, age):
        # rule: age must be 0 or more
        if age >= 0:
            self.age = age          # valid → keep it
        else:
            self.age = 0            # invalid → safe fallback

        self.name = name
```

So the flow is:

age input → check (>= 0?) → if yes: use it, if no: replace with 0 → object starts in a valid state

Quick mental check 🔍 (don’t code yet):

If you create:

```
pet1 = Pet("Buddy", 3)
pet2 = Pet("Ghost", -4)
```

What do you expect pet1.age and pet2.age to be after this?

Answer in words like: “pet1 age = …, pet2 age = …”

pet1 age=3, pet2 age=0

Perfect ✅

pet1.age = 3 (valid, so we keep it)
pet2.age = 0 (invalid -4, so we replace with safe fallback 0)

That’s exactly constructor validation + safe fallback in

action.

Next step: I want you to implement this yourself in the editor so it really sticks 💪

Here’s what to do:

Open the code editor for this lesson.
Create a Pet class with __init__(self, name, age) that:
if age >= 0, stores the real age
otherwise, stores 0
In the editor, create and print ages for:
one pet with a valid age (like 3)
one pet with a negative age (like -2)

Run, then Submit code.

Once you’ve submitted, I’ll look at it and we’ll build from there 🎯

Python
```
class Pet:
    def __init__(self, name, age):
        # rule: age must be 0 or more
        self.age = age if age > 0 else 0
        self.name = name
pet1 = Pet("Dunder", 3)
pet2 = Pet("RestInPieces", -5)
print(f'{pet1.name} is {pet1.age} years old')
print(f'{pet2.name} is {pet2.age} years old')
```
Nice work, Ron 🎯

You’ve already done constructor validation and a safe fallback:

Valid age 3 → kept as 3
Invalid age -5 → replaced with 0

One tiny thing: your rule is age > 0, so an age of exactly 0 would become 0 anyway (same result). Often for pets, 0 is a valid age (newborn), so using >= 0 is a bit clearer, but your behavior is still safe.

Now I want to add one more layer: thinking about object state.

A valid object state means all attributes follow the rules of the class when the object is created.

In your Pet, the rule is basically:

“age must never be negative once the object exists.”

Your validation guarantees that 👍

Does that idea of “valid object state = no impossible values inside the object” feel clear?

If yes, I’ll show you a quick visual of the flow next (input → check → accept or fallback).

true thiugh if zero gets passed init ends up zero passig or failing

Yeah, you’re thinking about it in the right way 👌

With your current code:

Python
self.age = age if age > 0 else 0
If age is 3 → self.age = 3
If age is −5 → self.age = 0
If age is 0 → self.age = 0 (because the condition age > 0 is false, so it goes to the else 0)

So 0 still ends up as 0 — it’s treated like an invalid value but the fallback is also 0, so the final stored value is fine.

If you want the rule to say clearly “0 is allowed”, you could change it to:

Python

```
self.age = age if age >= 0 else 0
Functionally, both keep the object in a valid state (no negatives). The second one just states the rule more directly.

Want to try a small twist next: add another validation, like making sure the pet’s name isn’t empty?

Python
class Pet:
    def __init__(self, name, age):
        # rule: age must be 0 or more
        self.age = age if age >= 0 else 0
        self.name = name if len(name) else "UnKnown"
pet1 = Pet("Dunder", 3)
pet2 = Pet("RestInPieces", -5)
pet3 = Pet("", 7)
print(f'{pet1.name} is {pet1.age} years old')
print(f'{pet2.name} is {pet2.age} years old')
print(f'{pet3.name} is {pet3.age} years old')
Take your time with the question below 👇

A pet’s constructor receives an invalid age of -3. What should responsible validation do?


A.
Store -3 unchanged


B.
Replace it with a safe fallback such as 0


C.
Add the age to the pet’s name


D.
Introduce inheritance

Nice job, Ron 🎉


You nailed the key idea:
```

When a constructor receives an invalid age like -3, responsible validation replaces it with a safe fallback, such as 0, instead of storing the bad value.

Why this is correct:

Storing -3 would give the object an impossible state (a pet can't be -3 years old).
Adding the age to the name or "introducing inheritance" doesn't solve the real problem: keeping the data meaningful and safe.
Using a fallback like 0 keeps the object usable and consistent with the class's rules.

In your own code, you did exactly this with ages and even extended it to names — that's responsible object design in practice 🛡️

You've mastered for this lesson:

Adding validation logic inside __init__
Using safe fallback values
Thinking about valid vs. invalid object state in real-world terms

If you're ready, you can jump to the next lesson in the Learn tab and keep building from here 🚀

How would you rate this lesson?

just adding i see a case for not always setting a default value on edge cases, say a financial record being recorded in a self printing tax class. here it my be prudent to toss an Assert error and not put a bad record value in

You're thinking like a real engineer here, Ron 🔥

You're absolutely right:

there are cases where silently fixing bad data with a

fallback is not appropriate.

Your example is spot on:

In a tax or financial system, a “self-printing tax class” storing money amounts
A bad record (wrong amount, corrupted data, etc.)
Instead of quietly doing something like amount = 0.0, you’d often want to:
refuse to create the object, or
raise an error (like an assertion or exception) so the issue is visible and can be handled

That’s a different strategy from what we practiced here, but it’s the same core idea of responsible object design:

Either:

fix invalid data safely (fallback) or
fail loudly and don’t let bad data into the system at all

In this lesson, we focused only on the “fix with fallback” path and avoided exceptions on purpose, but your mental model is ahead of that 👍

If you’d like, in a later lesson we can explore:

when to auto-correct with fallbacks
when to raise and stop everything
how that changes depending on domain (pets vs. taxes)

Want to keep going with more validation patterns, or ready to move on to the next lesson in the track?

```
class Pet:
    def __init__(self, name, age):
        # rule: age must be 0 or more
        self.age = age if age >= 0 else 0
```

```
        self.name = name if len(name) else "UnKnown"
pet1 = Pet("Dunder", 3)
pet2 = Pet("RestInPieces", -5)
pet3 = Pet("", 7)
print(f'{pet1.name} is {pet1.age} years old')
print(f'{pet2.name} is {pet2.age} years old')
print(f'{pet3.name} is {pet3.age} years old')

```